

Vítejte u druhého projektu do SUI. V rámci projektu Vás čeká několik cvičení, v nichž budete doplňovat poměrně malé fragmenty kódu (místo je vyznačeno pomocí `None` nebo `pass`). Pokud se v buňce s kódem již něco nachází, využijte/neničte to. Buňky nerušte ani nepřidávejte. Snažte se programovat hezky, ale jediná skutečně aktivně zakázaná, vyhledávaná a -- i opakovaně -- postihovaná technika je cyklení přes data (ať už explicitním cyklem nebo v rámci `list/dict` comprehension), tomu se vyhýbejte jako čert kříží a řešte to pomocí vhodných operací lineární algebry.

Až budete s řešením hotovi, vyexportujte ho ("Download as") jako PDF i pythonovský skript a ty odevzdejte pojmenované názvem týmu (tj. loginem vedoucího). Dbejte, aby bylo v PDF všechno vidět (nezůstal kód za okrajem stránky apod.).

U všech cvičení je uveden orientační počet řádků řešení. Berte ho prosím opravdu jako orientační, pozornost mu věnujte, pouze pokud ho významně překračujete.

```
import numpy as np
import copy
import matplotlib.pyplot as plt
import scipy.stats
from sklearn.metrics import accuracy_score
```

Přípravné práce

Prvním úkolem v tomto projektu je načíst data, s nimiž budete pracovat. Vybudujte jednoduchou třídu, která se umí zkonstruovat z cesty k negativním a pozitivním příkladům, a bude poskytovat:

- pozitivní a negativní příklady (`dataset.pos`, `dataset.neg` o rozměrech $[N, 7]$)
- všechny příklady a odpovídající třídy (`dataset.xs` o rozměru $[N, 7]$, `dataset.targets` o rozměru $[N]$)

K načítání dat doporučujeme využít `np.loadtxt()`. Netrapte se se zapouzdřováním a gettery, berte třídu jako Plain Old Data.

Načtěte trénovací (`{positives,negatives}.trn`), validační (`{positives,negatives}.val`) a testovací (`{positives,negatives}.tst`) dataset, pojmenujte je po řadě `train_dataset`, `val_dataset` a `test_dataset`.

(6 řádků)

```
class BinaryDataset:
    def __init__(self, path_to_positives, path_to_negatives):
        # Načtěte data ze souborů
        self.pos = np.loadtxt(path_to_positives)
        self.neg = np.loadtxt(path_to_negatives)

        self.xs = np.vstack((self.pos, self.neg))
        self.targets = np.hstack((np.ones(len(self.pos)),
np.zeros(len(self.neg))))
```

```

train_dataset = BinaryDataset('positives.trn', 'negatives.trn')
val_dataset = BinaryDataset('positives.val', 'negatives.val')
test_dataset = BinaryDataset('positives.tst', 'negatives.tst')

print('positives', train_dataset.pos.shape)
print('negatives', train_dataset.neg.shape)
print('xs', train_dataset.xs.shape)
print('targets', train_dataset.targets.shape)

positives (2280, 7)
negatives (6841, 7)
xs (9121, 7)
targets (9121,)

```

V řadě následujících cvičení budete pracovat s jedním konkrétním příznakem. Naimplementujte proto funkci, která vykreslí histogram rozložení pozitivních a negativních příkladů z jedné sady. Nezapomeňte na legendu, ať je v grafu jasné, které jsou které. Funkci zavoláte dvakrát, vykreslete histogram příznaku 5 -- tzn. šestého ze sedmi -- pro trénovací a validační data

(5 řádků)

```

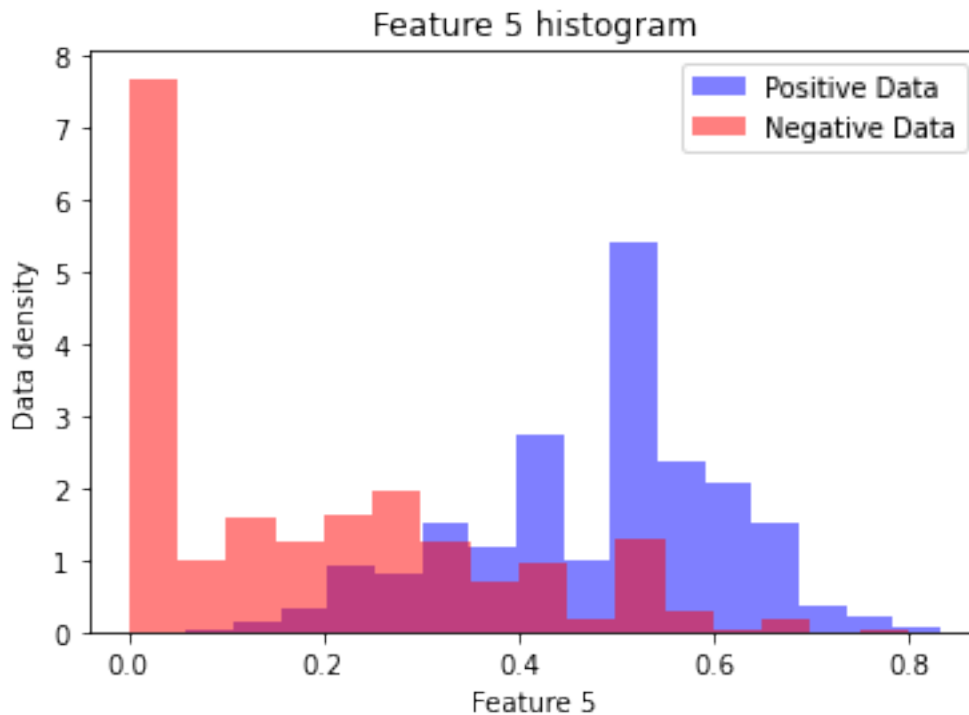
FOI = 5 # Feature Of Interest

def plot_dataset():
    plt.hist(train_dataset.pos[:, FOI], bins=16, density=True,
color='blue', alpha=0.5, label='Positive Data')
    plt.hist(train_dataset.neg[:, FOI], bins=16, density=True,
color='red', alpha=0.5, label='Negative Data')

def plot_data(title_, xlabel_, ylabel_):
    plt.title(title_)
    plt.xlabel(xlabel_)
    plt.ylabel(ylabel_)
    plt.legend(loc='upper right')
    plt.show()

plot_dataset()
plot_data(title_=f'Feature {FOI} histogram', xlabel_=f'Feature {FOI}',
ylabel_='Data density')

```



Evaluace klasifikátorů

Než přistoupíte k tvorbě jednotlivých klasifikátorů, vytvořte funkci pro jejich vyhodnocování. Nechť se jmenuje `evaluate` a přijímá po řadě klasifikátor, pole dat (o rozměrech $[N, F]$) a pole tříd ($[N]$). Jejím výstupem bude *přesnost* (accuracy), tzn. podíl správně klasifikovaných příkladů.

Předpokládejte, že klasifikátor poskytuje metodu `.prob_class_1(data)`, která vrácí pole posteriorních pravděpodobností třídy 1 pro daná data. Evaluační funkce bude muset provést tvrdé prahování (na hodnotě 0.5) těchto pravděpodobností a srovnání získaných rozhodnutí s referenčními třídami. Využijte fakt, že `numpy`ovská pole lze mj. porovnávat se skalárem.

(3 řádky)

```
def evaluate(classifier, inputs, targets):
    probs = classifier.prob_class_1(inputs) # Obtaining class 1
    probabilities from a classifier for given data
    predictions = (probs >= 0.5).astype(int) # Probability
    thresholding (0.5 is the threshold)

    return accuracy_score(targets, predictions)

class Dummy:
    def prob_class_1(self, xs):
        return np.asarray([0.2, 0.7, 0.7])

print(evaluate(Dummy(), None, np.asarray([0, 0, 1]))) # should be
0.66
```

```
0.6666666666666666
```

Baseline

Vytvořte klasifikátor, který ignoruje vstupní data. Jenom v konstruktoru dostane třídu, kterou má dávat jako tip pro libovolný vstup. Nezapomeňte, že jeho metoda `.prob_class_1(data)` musí vracet pole správné velikosti.

(4 řádky)

```
class PriorClassifier:
    def __init__(self, prior_class):
        self.prior_class = prior_class

    def prob_class_1(self, data):
        return np.full(len(data), self.prior_class)  # Returns an
array of class 1 probabilities of the same length as the data, but
always containing the prior_class value
```

```
baseline = PriorClassifier(0)
val_acc = evaluate(baseline, val_dataset.xs[:, FOI],
val_dataset.targets)
print('Baseline val acc:', val_acc)
```

```
Baseline val acc: 0.75
```

Generativní klasifikátory

V této části vytvoříte dva generativní klasifikátory, oba založené na Gaussovu rozložení pravděpodobnosti.

Začněte implementací funce, která pro daná 1-D data vrátí Maximum Likelihood odhad střední hodnoty a směrodatné odchylky Gaussova rozložení, které data modeluje. Funkci využijte pro natrénování dvou modelů: pozitivních a negativních příkladů. Získané parametry -- tzn. střední hodnoty a směrodatné odchylky -- vypíšete.

(1 řádek)

```
def mle_gauss_1d(data):
    return np.mean(data), np.std(data)

mu_pos, std_pos = mle_gauss_1d(train_dataset.pos[:, FOI])
mu_neg, std_neg = mle_gauss_1d(train_dataset.neg[:, FOI])

print('Pos mean: {:.2f} std: {:.2f}'.format(mu_pos, std_pos))
print('Neg mean: {:.2f} std: {:.2f}'.format(mu_neg, std_neg))
```

Pos mean: 0.48 std: 0.13
Neg mean: 0.17 std: 0.18

Ze získaných parametrů vytvořte `scipy`ovská gaussovská rozložení `scipy.stats.norm`. S využitím jejich metody `.pdf()` vytvořte graf, v němž srovnáte skutečné a modelové rozložení pozitivních a negativních příkladů. Rozsah x-ové osy volte od -0.5 do 1.5 (využijte `np.linspace`) a u volání `plt.hist()` nezapomeňte nastavit `density=True`, aby byl histogram normalizovaný a dal se srovnávat s modelem.

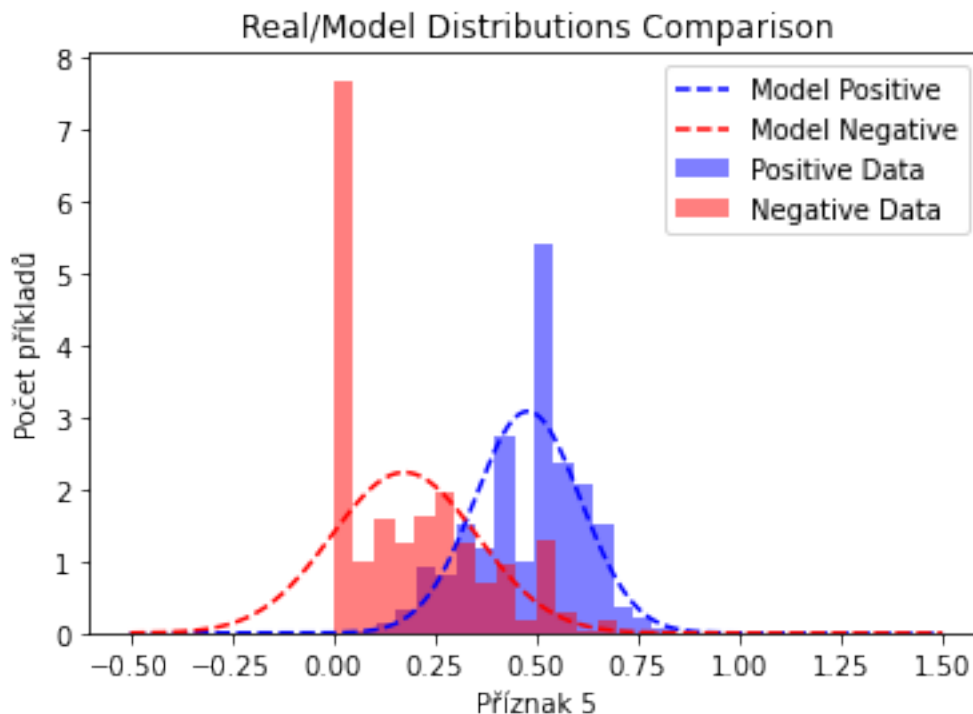
(2 + 8 řádků)

```
x_range = np.linspace(-0.5, 1.5, 1000) # The x-axis range

# Model Gaussian distributions
model_pos = scipy.stats.norm.pdf(x_range, mu_pos, std_pos)
model_neg = scipy.stats.norm.pdf(x_range, mu_neg, std_neg)

# Render model layouts
plt.plot(x_range, model_pos, color='blue', linestyle='--',
label='Model Positive')
plt.plot(x_range, model_neg, color='red', linestyle='--', label='Model
Negative')

plot_dataset()
plot_data(title_='Real/Model Distributions Comparison',
xlabel_=f'Příznak {FOI}', ylabel_='Počet příkladů')
```



Naimplementujte binární generativní klasifikátor. Při konstrukci přijímá dvě rozložení poskytující metodu `.pdf()` a odpovídající apriorní pravděpodobnost tříd. Dbejte, aby Vám uživatel nemohl zadat neplatné apriorní pravděpodobnosti. Jako všechny klasifikátory v tomto projektu poskytuje metodu `prob_class_1()`.

(9 řádků)

```
class GenerativeClassifier2Class:
    def __init__(self, model_pos, model_neg, prior_pos, prior_neg):
        if not ((0 <= prior_pos <= 1) and (0 <= prior_neg <= 1)): #
            Validation of a prior probabilities
            raise ValueError("Prior probabilities must be in the range
<0, 1>.")

        self.model_pos = model_pos
        self.model_neg = model_neg
        self.prior_pos = prior_pos
        self.prior_neg = prior_neg

    def prob_class_1(self, data):      # Computation of class 1
        posterior probabilities
        likelihood_pos = scipy.stats.norm.pdf(data, loc=mu_pos,
scale=std_pos)
        likelihood_neg = scipy.stats.norm.pdf(data, loc=mu_neg,
scale=std_neg)

        posterior_positive = (self.prior_pos * likelihood_pos) /
((self.prior_pos * likelihood_pos) + (self.prior_neg *
likelihood_neg))
        return posterior_positive
```

Nainstancujte dva generativní klasifikátory: jeden s rovnoměrnými priory a jeden s apriorní pravděpodobností 0.75 pro třídu 0 (negativní příklady). Pomocí funkce `evaluate()` vyhodnotíte jejich úspěšnost na validačních datech.

(2 řádky)

```
# classifier with equal prior probabilities (0.5 for both classes)
classifier_flat_prior = GenerativeClassifier2Class(model_pos,
model_neg, 0.5, 0.5)

# classifier with a prior probability of 0.75 for class 0 (negative
examples)
classifier_full_prior = GenerativeClassifier2Class(model_pos,
model_neg, 0.25, 0.75)

# Evaluation of accuracy on validation data
print('flat:', evaluate(classifier_flat_prior, val_dataset.xs[:, FOI],
val_dataset.targets))
```

```
print('full:', evaluate(classifier_full_prior, val_dataset.xs[:, F0I],
val_dataset.targets))

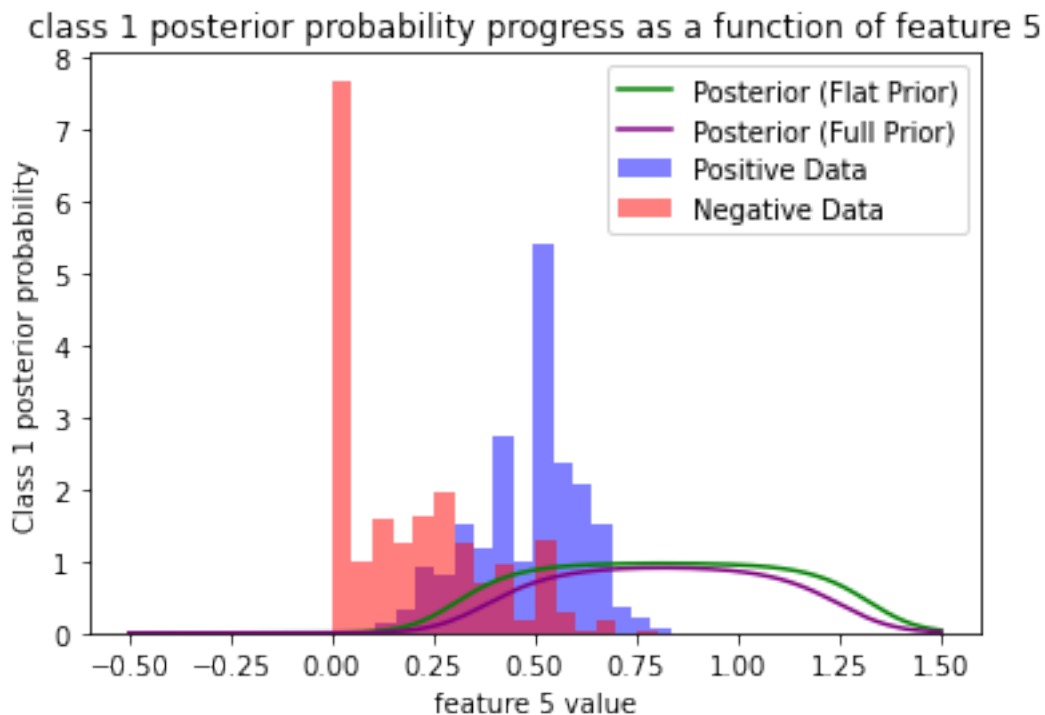
flat: 0.809
full: 0.8475
```

Vykreslete průběh posteriorní pravděpodobnosti třídy 1 jako funkci příznaku 5, opět v rozsahu <-0.5; 1.5> pro oba klasifikátory. Do grafu zakreslete i histogramy rozložení trénovacích dat, opět s `density=True` pro zachování dynamického rozsahu.

(8 řádků)

```
def plot_posterior(classifier, clr, lbl):
    posterior = classifier.prob_class_1(x_range)
    plt.plot(x_range, posterior, color=clr, label=lbl)

plot_posterior(classifier_flat_prior, clr='green', lbl='Posterior
(Flat Prior)')
plot_posterior(classifier_full_prior, clr='purple', lbl='Posterior
(Full Prior)')
plot_dataset()
plot_data(title_=f'class 1 posterior probability progress as a
function of feature {F0I}', xlabel_=f'feature {F0I} value',
ylabel_='Class 1 posterior probability')
```



Diskriminativní klasifikátory

V následující části budete pomocí (lineární) logistické regrese přímo modelovat posteriorní pravděpodobnost třídy 1. Modely budou založeny čistě na NumPy, takže nemusíte instalovat nic dalšího. Nabitějších toolkitů se dočkáte ve třetím projektu.

```
def logistic_sigmoid(x):
    return np.exp(-np.logaddexp(0, -x))

def binary_cross_entropy(probs, targets):
    probs = np.clip(probs, 1e-15, 1 - 1e-15) # Clip probabilities to
    avoid extremes
    return np.sum(-targets * np.log(probs) - (1-targets)*np.log(1-
    probs))

class LogisticRegressionNumpy:
    def __init__(self, dim):
        self.w = np.array([0.0] * dim)
        self.b = np.array([0.0])

    def prob_class_1(self, x):
        return logistic_sigmoid(x @ self.w + self.b)
```

Diskriminativní klasifikátor očekává, že dostane vstup ve tvaru `[N, F]`. Pro práci na jediném příznaku bude tedy zapotřebí vyřezávat příslušná data v správném formátu `[N, 1]`. Doimplementujte třídu `FeatureCutter` tak, aby to zařizovalo volání její instance. Který příznak se použije, nechť je konfigurováno při konstrukci.

Může se Vám hodit `np.newaxis`.

(2 řádky)

```
class FeatureCutter:
    def __init__(self, fea_id):
        self.fea_id = fea_id

    def __call__(self, x):
        return x[self.fea_id, np.newaxis] if len(x.shape) == 1 else
        x[:, self.fea_id, np.newaxis]
```

Dalším krokem je implementovat funkci, která model vytvoří a natrénuje. Jejím výstupem bude (1) natrénovaný model, (2) průběh trénovací loss a (3) průběh validační přesnosti. Neuvažujte žádné minibatche, aktualizujte váhy vždy na celém trénovacím datasetu. Po každém kroku vyhodnoťte model na validačních datech. Jako model vracejte ten, který dosáhne nejlepší validační přesnosti. Jako loss použijte binární cross-entropii a logujte průměr na vzorek. Pro výpočet validační přesnosti využijte funkci `evaluate()`. Oba průběhy vracejte jako obyčejné seznamy.

(cca 11 řádků)


```
def train_logistic_regression(epochs, lr, in_dim, fea_preprocessor):
    model = LogisticRegressionNumpy(in_dim)
    best_model = copy.deepcopy(model)
    losses = []
    accuracies = []

    train_X = fea_preprocessor(train_dataset.xs)
    train_t = train_dataset.targets
    val_X = fea_preprocessor(val_dataset.xs)
    val_t = val_dataset.targets

    for epoch in range(epochs):
        probs = model.prob_class_1(train_X)
        loss = binary_cross_entropy(probs, train_t)
        losses.append(np.mean(loss))
        val_acc = evaluate(model, val_X, val_t)
        accuracies.append(val_acc)
        gradient_w = train_X.T @ (probs - train_t)
        gradient_b = np.sum(probs - train_t)
        model.w -= lr * gradient_w
        model.b -= lr * gradient_b
        # Porovnání s nejlepším modelem
        if val_acc > evaluate(best_model, val_X, val_t):
            best_model = copy.deepcopy(model)
    return best_model, losses, accuracies
```

Funkci zavolejte a natrénуйте model. Uvedte zde parametry, které vám dají slušný výsledek. Měli byste dostat přesnost srovnatelnou s generativním klasifikátorem s nastavenými priory. Neměli byste potřebovat víc, než 100 epoch. Vykreslete průběh trénovací loss a validační přesnosti, osu x značte v epochách.

V druhém grafu vykreslete histogramy trénovacích dat a pravděpodobnost třídy 1 pro x od -0.5 do 1.5, podobně jako výše u generativních klasifikátorů.

(1 + 5 + 8 řádků)

```
def plot_loss_and_accuracy(losses, accuracies, epochs):
    # Create figure and axis objects
    fig, ax1 = plt.subplots(figsize=(12, 4))

    # Plot Training Loss on the left y-axis
    ax1.set_xlabel('Epochs')
    ax1.set_ylabel('Training Loss', color='tab:red')
    ax1.plot(range(epochs), losses, label='Training Loss',
            color='tab:red')
    ax1.tick_params(axis='y', labelcolor='tab:red')

    # Create a second y-axis for Validation Accuracy on the right side
    ax2 = ax1.twinx()
    ax2.set_ylabel('Validation Accuracy', color='tab:blue')
```

```

    ax2.plot(range(epochs), accuracies, label='Validation Accuracy',
color='tab:blue')
    ax2.tick_params(axis='y', labelcolor='tab:blue')

    # Title and legend
    plt.title('Training Loss and Validation Accuracy Over Epochs')
    plt.legend()
    plt.show()

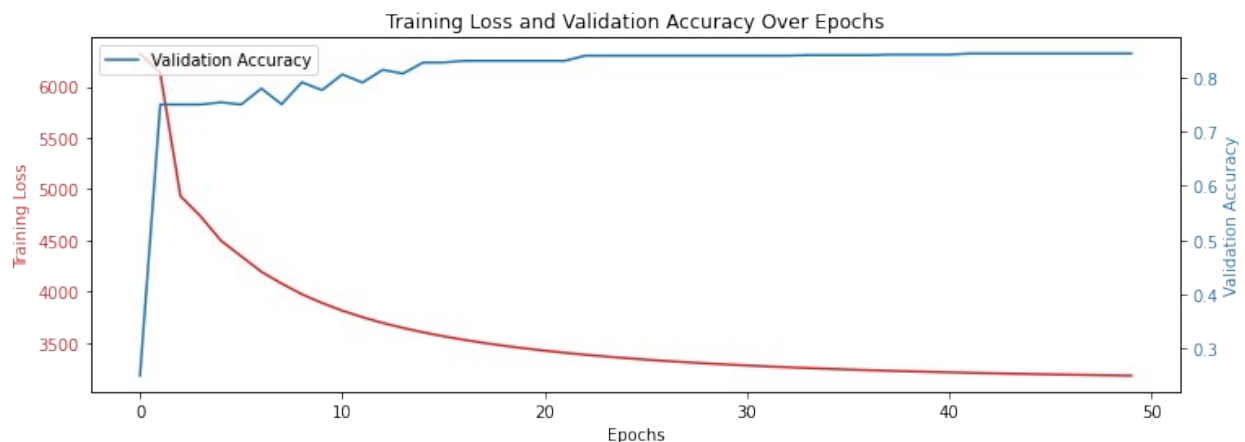
lr = 0.001
epochs_cnt = 50
model, losses, accuracies = train_logistic_regression(epochs_cnt, lr,
in_dim=1, fea_preprocessor=FeatureCutter(fea_id=F0I)) # Trénování
logistické regrese

plot_loss_and_accuracy(losses, accuracies, epochs_cnt)

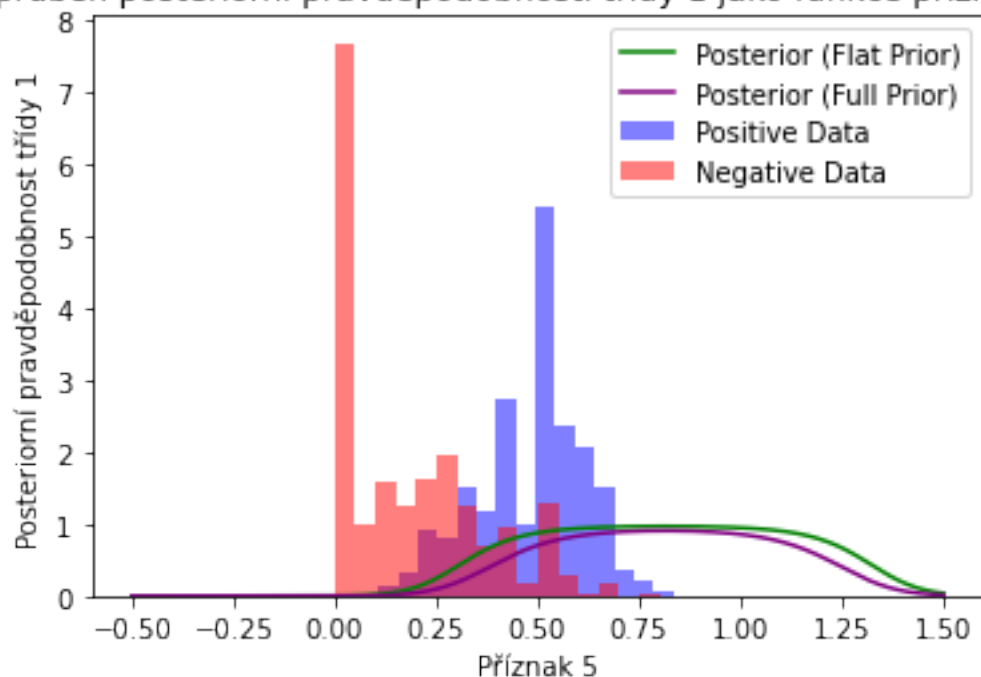
plot_posterior(classifier_flat_prior, clr='green', lbl='Posterior
(Flat Prior)')
plot_posterior(classifier_full_prior, clr='purple', lbl='Posterior
(Full Prior)')
plot_dataset()
plot_data(title_='průběh posteriorní pravděpodobnosti třídy 1 jako
funkce příznaku 5', xlabel_=f'Příznak {F0I}', ylabel_='Posteriorní
pravděpodobnost třídy 1')

print('w', model.w.item(), 'b', model.b.item())
print('final accuracy:', evaluate(model, val_dataset.xs[:, F0I][:,
np.newaxis], val_dataset.targets))

```



průběh posteriorní pravděpodobnosti třídy 1 jako funkce příznaku 5



w 7.283808661174657 b -3.4184677292184307
final accuracy: 0.844

Všechny vstupní příznaky

V posledním cvičení natrénujete logistickou regresi, která využije všech sedm vstupních příznaků. Zavolejte funkci z předchozího cvičení, opět vykreslete průběh trénovací loss a validační přesnosti. Měli byste se dostat nad 90 % přesnosti.

Může se Vám hodit `lambda` funkce.

(1 + 5 řádků)

```
epochs_cnt = 250
model_all, losses, accuracies = train_logistic_regression(epochs_cnt,
lr=0.000001, in_dim=train_dataset.xs.shape[1], fea_preprocessor=lambda
x: x)
plot_loss_and_accuracy(losses, accuracies, epochs_cnt)
print('Validation Accuracy:', evaluate(model_all, val_dataset.xs,
val_dataset.targets))
```



Validation Accuracy: 0.944

Závěrem

Konečně vyhodnoťte všech pět vytvořených klasifikátorů na testovacích datech. Stačí doplnit jejich názvy a předat jim odpovídající příznaky. Nezapomeňte, že u logistické regrese musíte zopakovat formátovací krok z FeatureCutteru.

```
xs_full = test_dataset.xs
xs_foi = test_dataset.xs[:, FOI][:, np.newaxis]
targets = test_dataset.targets

print('Baseline:', evaluate(baseline, xs_full, targets))
print('Generative classifier (w/o prior):',
evaluate(classifier_flat_prior, xs_foi, targets))
print('Generative classifier (correct):',
evaluate(classifier_full_prior, xs_foi, targets))
print('Logistic regression:', evaluate(model, xs_foi, targets))
print('Logistic regression all features:', evaluate(model_all,
xs_full, targets))
```

```
Baseline: 0.75
Generative classifier (w/o prior): 0.8
Generative classifier (correct): 0.847
Logistic regression: 0.853
Logistic regression all features: 0.938
```

Blahopřejeme ke zvládnutí projektu! Nezapomeňte spustit celý notebook načisto (Kernel -> Restart & Run all) a zkontrolovat, že všechny výpočty prošly podle očekávání.

Mimochodem, vstupní data nejsou synteticky generovaná. Nasbírali jsme je z baseline řešení historicky prvního SUI projektu; vaše klasifikátory v tomto projektu predikují, že daný hráč vyhraje dicewars, takže by se daly použít jako heuristika pro ohodnocování listových uzlů ve stavovém prostoru hry. Pro představu, data jsou z pozic pět kol před koncem partie pro daného

hráče. Poskytnuté příznaky popisují globální charakteristiky stavu hry jako je například poměr délky hranic předmětného hráče k ostatním hranicím. Nejeden projekt v ročníku 2020 realizoval požadované "strojové učení" kopií domácí úlohy.

